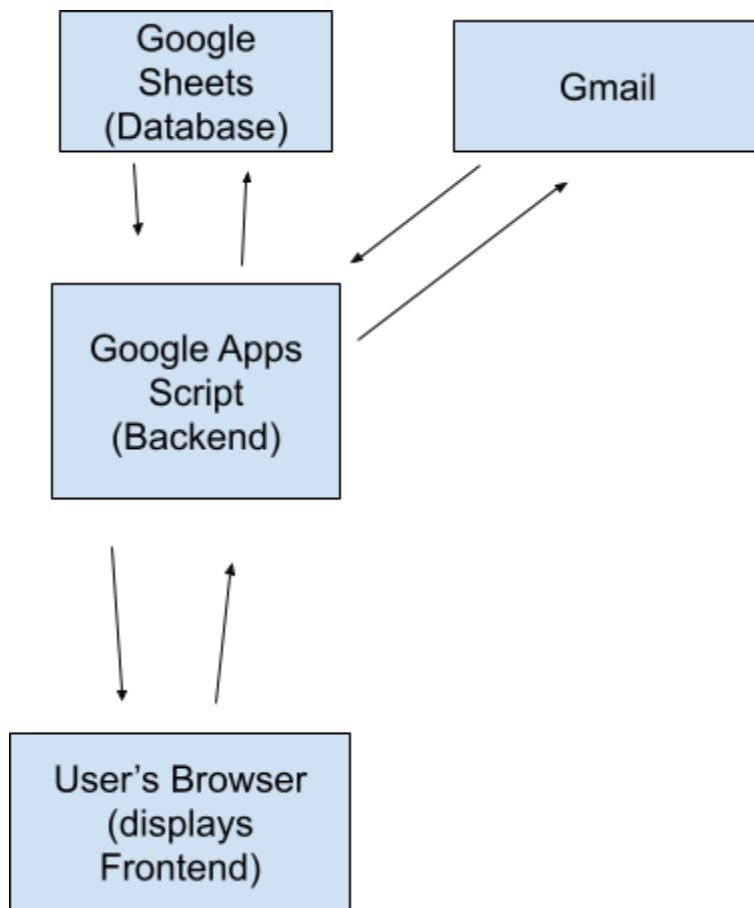


Equipment Checkout System

Code Navigation and Architecture Guide

Audience: New owners and beginner maintainers who need to understand how the code is organized, how data flows through the system, and where to make changes safely.

General Architecture



Gmail:

- Used for sending emails as a rental receipt as the role of the current teacher.
 - For instance, the program will send the email under the current teacher operating the system and to: the email to the current student.
 - Is based on the template of '*s[id]@domain.org*'
 - There is a limit of how many emails can be sent per day ([1500](#)).

Google Sheets (Database)

- The system stores Student, Item, and Rental info in a Google Sheets Document specified by the user during setup instead of a traditional database. This made initial implementation easier and keeps the data in a way that it can be edited by teachers directly if the system ever fails or if they just want to. Also, this should make it easier for teachers to migrate to another system (or just use the spreadsheet and manual entry) in case the system ever fails.

Google Apps Script (Backend)

- Used in place of a standard hosting provider
- Reasons why this was used
 - Data stays within the district's existing Google Workspace and Google handles user accounts and the system should only be available to users once Google checks they are authorized. This reduces maintenance since if we had hosted it somewhere else and exposed it to the whole internet we might have had to deal with security updates and user authorization. (there are probably other options: there are probably ways we could have still had a hosting company handle the authorization but we just used Google Apps Script and had Google handle sign in, other options should be considered if Google Apps Script is no longer an option)

User's Browser (displays Frontend)

- Browses to the URL of the system
 - This sends an HTTP GET request to the backend (Google Apps Script) which is handled by the `doGet` function in Code.gs
- Displays the user interface

File Structure

- Code.gs
 - Backend Code
 - Written in what is basically just JS + some google APIs
- index.html
 - Main frontend code
 - Includes
 - HTML
 - Inline JS
 - Inline CSS
- setup.html
 - Similar to index.html but just shown to the user during initial setup to ask them for the URL to the Google Sheets document which will be used as the database
- 3rd_party_library_<LIBRARY-NAME>.html
 - These files hold 3rd party dependencies
 - This structure is used instead of just using src in <script> tags and a similar thing for <style> tags. In case a CDN we would have used instead ever goes down (probably unlikely, but just in case) this way the dependencies are sent to the client browser's from the same place as the code (and the versions stay the same).
 - Used by *include3rdPartyLibrary()* in Code.gs
- 3rd_party_library_template.html
 - Template for the 3rd party library inclusion. See above.

Frontend Design

3 Tabs/Sections

Name	Description	ID	Related Functions
Items	Show a list of items stored in the database	items-tab	showSection() renderItems()
Rentals	Store a list of current rentals (Does not show returned rentals, those are only shown on the Google Sheets spreadsheet)	rentals-tab	showSection() renderRentals()
Students	Show a list of students stored in the database	people-tab	showSection() renderStudents()

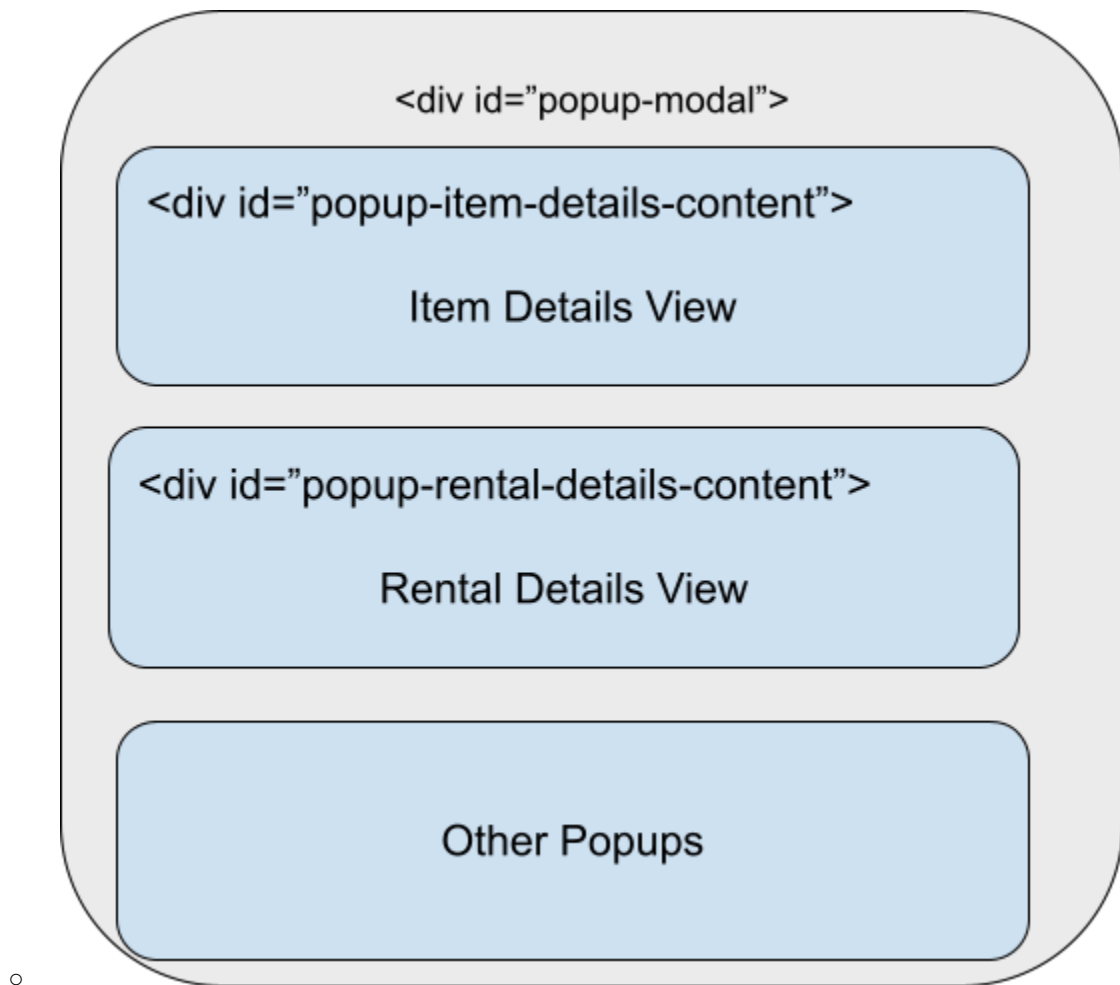
Popups

- Press Control + F and search the “ID” to find the specific table.

Name	Description	ID	Related Functions
Item Details	View all data on an item and be able to edit it.	popup-item-details-content	<i>handleDelete()</i> <i>openItemDetails()</i>
View Rental Details	View all data on a rental.	popup-rental-details-content	<i>openRentalDetails()</i>
Student Details	View all data on a student and be able to edit it.	popup-student-details-content	<i>handleDelete()</i> <i>openStudentDetails()</i>
Add Items	Popup for adding up to 20 items at a time.	popup-add-items-content	<i>updateAddItemsPopupRows()</i> <i>setallschool()</i> <i>updateAddItemsPopupRows()</i> <i>openAddItemsView()</i>
Add Students	Popup for adding up to 20 students at a time.	popup-add-students-content	<i>updateAddStudentsPopupRows()</i> <i>openAddStudentsView()</i>
Scanning Mode	Popup for checking items in and out.	scanning-mode-popup	<i>submitScanningMode()</i> <i>scanningModeSelectMode(string mode)</i> <i>closeScanningMode()</i> <i>openScanningMode()</i>

			<i>openMakeRentalView()</i>
Report Mode (Currently hidden from users)	Popup for generating reports about data collected by the system.	reports-view-popup	openReportsView()

Structured Like This:



How do the popups work?

- To Open a popup
 - User clicks a button which calls an “open” Function
 - List of open functions:
 - `openMakeRentalView()`

- openAddStudentsView()
- openItemDetails()
- Etc. (ctrl-f through index.html with the search term “open” to find them)
- The open function may add data to it’s specific popup or or run other steps but then they all call:
 - document.getElementById(“<ID OF SPECIFIC POPUP HERE”).classList.remove(“hidden”)
 - This unhides the specific popup, but the popup is still not visible because the <div> with id=“popup-modal” is still hidden. So..
 - document.getElementById(“popup-modal”).classList.remove(“hidden”) - to unhide the parent container of all the popups
 - *disablePageScroll()*
- Popup provides HTML elements and associated functions to edit or modify data. Ex. *saveAddItemsView()*
- To Close a popup
 - User clicks a close button
 - Calls *closeModalDialog()*
 - Or a separate close function which will also eventually probably call *closeModalDialog()* as well as possibly do other tasks specific to that close function.

How to call a backend function?

To call a backend function from the frontend do the following

```
google.script.run.withSuccessHandler((data) => {

    dataReturnedFromBackendFunction = JSON.parse(data) //this JSON.parse(data)
may not be necessary, it just depends on what format the backend function returns in

    //use the returned data here

}).<backendFunctionNameHere>(<arguments here>);
```

Good References

- [Official Google Apps Script Functions Documentation](#)
- [Mozilla Javascript \(JS\) Reference](#)
- [W3Schools](#) - HTML, CSS, JS guides + many more not relevant to this project

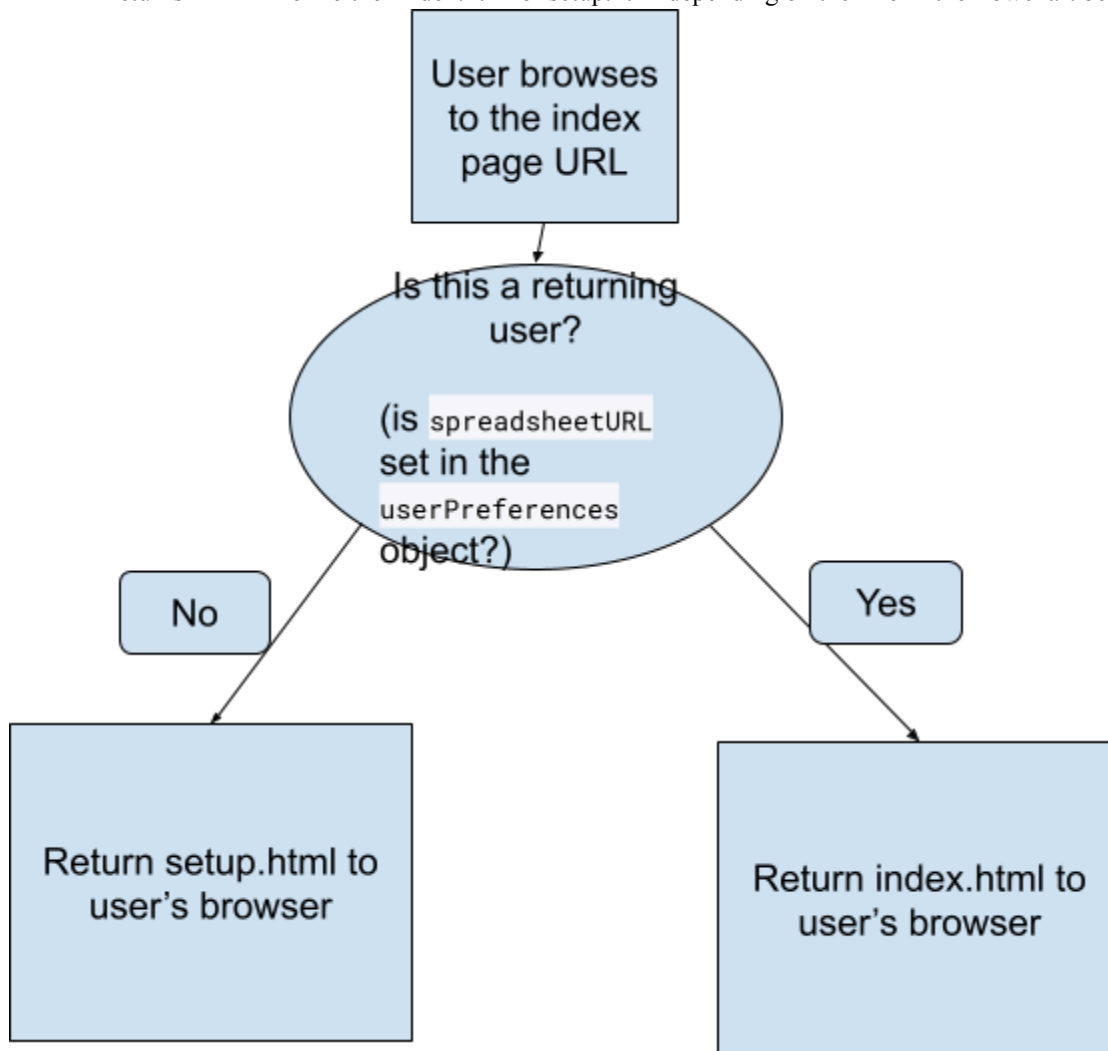
Debugging

- If error is on frontend:
 - 1. Take error number from Browser JS console.
 - 2. Add 21 (do not know why this number specifically, just figured this out by deliberately causing an error on a known line number then calculating this)
 - 3. Add the line of the <script> tag of the JS section where the error is occurring. (As of writing this (7-14-2026) this line number is 1454
 - 4. Look in index.html using this line number to find the line of code throwing the error.
- If error is on backend:
 - View execution logs under “Executions” on Google Apps Script project

Backend Functions (Code.gs):

doGet()

- Handles incoming HTTP GET requests from user's browsers
- Returns HTML from either index.html or setup.html depending on the info in the flowchart below:



del()

- Forgets which spreadsheet the user selected as the database

setInventoryURL()

- Stores the URL to the Google Sheets file the user indicated should be used as the database

resetInventoryURL()

- Basically the same as del()
- TODO: check why both of these exist

getSetupPage()

- Used by doGet
- Just reads setup.html and outputs it as an HTMLOutput object back to doGet()

include3rdPartyLibrary()

- Used by doGet to include 3rd party libraries (CSS or JS) as part of the HTML sent to the client.
- Called using statements like `<?!= include3rdPartyLibrary('flowbite_css'); ?>` inside of the .html files and replaced by the code from the library when .evaluate() is called on a template. For example: `createTemplateFromFile('index').evaluate()`. When this is called, everything between `<?!=` and `?>` is replaced with the result of the function call to `include3rdPartyLibrary()`

addStudentsToDatabase(students)

- Adds new student entries to the Google Sheets database spreadsheet
- The parameter students is of the following format: `[{"id": "ID1", "classYear": "CLASS_YEAR1", "period": "PERIOD1", "name": "NAME1"}, {"id": "ID2", "classYear": "CLASS_YEAR2", "period": "PERIOD2", "name": "NAME2"}...]`

addItemToDatabase(items)

- Adds new item entries to the database
- The items parameter is of the following format: `[{roomArea: "C999", itemDescription: "item description here", makeModel: "brand", assetTagNumber: "EX999", serialNumber: "SN999", datePurchased: "2033-2034", purchaseOrder: "", fund: "", cost: "999.99", notes: "this is a test item", status: "In", rentalID: ""}]`

editItemsInDatabase(items)

- Edits existing items in the database
- The items parameter is the same format as *addItemToDatabase()*

editStudentsInDatabase(students)

- Edits existing students in the database
- The students parameter is the same format as *addStudentsToDatabase()*

deleteItemsFromInventory(string[] assetTags)

- Delete items from the database given a list of assetTags
- Checks to make sure the items given are not associated with any outstanding rentals before deletion

deleteItemFromInventory(string assetTag)

- Delete a single item in the database by calling deleteItemsFromInventory and passing just the single assetTag
- This function is redundant

deleteStudentsFromDatabase(ids)

- Delete students from database given a list of student IDs
- Checks to make sure the students are not associated with outstanding rentals before deletion

getInventoryItems()

- **Obsolete**

convertSchool(school)

- **OBSOLETE**

getStudents()

- Returns a dictionary of students indexed by studentID
- Structure of the dictionary:

```
{
    "STUDENT_1_ID":
        {
            "id", "STUDENT_1_ID",
            "classYear": "STUDENT_1_CLASS_YEAR",
            "period": "STUDENT_1_PERIOD",
            "name": "STUDENT_1_NAME"}
        },
    "STUDENT_2_ID":
        {
            ...
        },
    ...
}
```

getItems()

- Returns a dictionary of items indexed by Asset Tag
- Structure of the dictionary:

```
{
    "ITEM_1_ASSET_TAG":
        {
            "roomArea": "ITEM_1_ROOM_AREA",
            "itemDescription": "ITEM_1_ITEM_DESCRIPTION",
            "makeModel": "ITEM_1_MAKE_MODEL",
            "assetTagNumber": "ITEM_1_ASSET_TAG",
            "serialNumber": "ITEM_1_SERIAL_NUMBER",
            "notes": "ITEM_1_NOTES",
            "status": "ITEM_1_STATUS",
            "rentalID": "ITEM_1_RENTAL_ID",
            "school": "ITEM_1_SCHOOL",
        },
    ...
}
```

```

        "ITEM_2_ASSET_TAG":
        {
            ...
        },
        ...
    }
}

```

buildRentalReceiptHTML(`{ studentID, itemName, assetTag, rentalID, dateOut, dateToReturn, contactEmail }`)

- Builds a custom HTML webpage to be viewed on the email

createRental(`studentID, assetTag, dateToReturn`)

- Try to check out the item given by *assetTag* to the student given by *studentID* with the given *dateToReturn*
- Check that the *assetTag*, *studentID*, and *dateToReturn* are valid and that the item is not already checked out
- Try to send a receipt email to the student by building their email address using the *studentID*

getRentals()

- Return a dictionary of Rentals indexed by the Rental IDs
- Structure of the dictionary:

```

{
    "RENTAL_1_ID":
    {
        "rentalID": Number("RENTAL_1_ID"),
        "itemID": "RENTAL_1_ITEM_ID",
        "status": "RENTAL_1_STATUS",
        "checkedOutTo": "RENTAL_1_CHECKED_OUT_TO_STUDENT_ID",
        "dateCheckedOut": Date("RENTAL_1_DATE_CHECKED_OUT"),
        "dateExpectedReturned": Date("RENTAL_1_DATE_EXPECTED_TO_RETURN"),
        "dateReturned": Date("RENTAL_1_DATE_RETURNED"),
        "notes": "RENTAL_1_NOTES",
        "returnRental": Function\("RENTAL\_1\_RETURN\_FUNCTION"\),
    }
}

```

- **rental.returnRental(returnedBy)**

- Each rental has it's own function to return it which is stored in the dictionary of rentals returned by `getRentals()`
- To return the rental call `rentals["RENTAL_ID"].returnRental("STUDENT_ID_OF_STUDENT_WHO_IS_RETURNING_THE_ITEM")`; assuming the dictionary was stored into a variable called *rentals*
- **HOWEVER**, see *returnRentals()*, it calls this function and is what should be used. This function should only ever be called by the *returnRentals()* function

buildMultiRentalReceiptHTML(`{ rentals, contactEmail, anyLate }`)

- When the user returns their item, an email template gets sent out
- *Rentals* is current Rental (SINGULAR, I KNOW) that was made
- *contactEmail* is the teacher to reach out to with any questions
- *anyLate* is a boolean if the current date is past the return date

returnRentals(itemsToReturn, returnedBy)

- Returns the items listed in *itemsToReturn* (which is a list of assetTags), and indicate they were returned by the student listed with their student ID in *returnedBy*

getCodeType()

- Obsolete (moved to [index.html](#) / frontend)

generateReport(reportType, timespanStart, timespanEnd)

- Generates a PDF corresponding to the desired reportType
- The existing report types are item popularity and usage report (itemPopularityUsageReport), and a report type that lists lost items (lostItemReport)

doSomething()

- Just used to call backend functions for debugging
- Can put backend functions with test parameters in this function then run it using the Google App Script editor UI to call this function without opening the front end.

getSpreadsheetURL()

- Returns the URL to the Google Sheets document acting as the database

findMakeModelBasedOnAssetTag(assetTag)

- Return the make/model of an item given the *assetTag* of the item

Frontend Functions:

refresh()

- Refresh the information displayed to the user in the Items, Student, and Rental tables on the dashboard

generateBarcode(string data)

- Returns a PNG representation of a barcode. The barcode encodes the data stored in the “data” function argument.

generateBarcodePDF(info)

- Generate a PDF containing the info stored in info and prompt the user to save it to their computer
- Uses *generateBarcode()*

getSelectedItems()

- Returns a list of Asset Tags of items who have been selected on the dashboard (using the checkboxes)

generateItemBarcodes()

- Generate a PDF of all the selected items.
- Uses *generateBarcodePDF()*

generateStudentBarcodes()

- Generate a PDF of all the selected items.
- Uses *generateBarcodePDF()*

deleteSelectedItems()

- Prompts the user whether they really want to delete the selected items, then calls *deleteItemsFromInventory()* on the backend if confirmed

deleteSelectedStudents()

- Prompts the user whether they really want to delete the selected students, then calls *deleteStudentsFromDatabase()* on the backend if confirmed

deleteSelectedStudents()

- Prompts the user whether they really want to delete the selected students, then calls *deleteStudentsFromDatabase()* on the backend if confirmed

updateAddItemsPopupRows(numberOfItemsToAdd)

- Add *numberOfItemsToAdd* more rows to the [Add Items popup](#)
- Append the IDs to the global *addItemRowIDs* variable

resizeTextAreaToFitContent(id)

- Resize the height of the <textarea> HTML element given by *id* to be large enough to fit all the existing content

removeAddItemsPopupRow(id)

- Remove a row from the [Add Items popup](#)
- Remove the ID of this row from the global variable *addItemRowIDs*

saveAddItemsView()

- Save the items the user has added via the [Add Items popup](#) to the database, then generate a barcode PDF for the user and prompt them to save it
- First make sure each row has an asset tag, if not then highlight the invalid box and alert the user
- Also check that none of the new asset tags are duplicates of each other

saveAddStudentsView()

- Save the students the user has added via the Add Students popup to the database.
- First make sure that each row has a valid student ID (not a blank string), a valid period if a period was given (can be converted to a number) (a blank string is also valid), and a valid class year (same rules as for the period)
- Check that none of the new student IDs are duplicates of each other

saveModalEdits()

- Save edits made to an Item, Student, or Rental (if implemented in future, currently not implemented for Rentals) or save the new items or students.
- Determines whether to save edits to item, student, or rental, or to save new items or new students based on the current title of the popup-modal

setallschool()

- Sets all of the current items' school attribute to the first row's school attribute

closeModalDialog()

- Closes the popup-modal div and all the child popups within it.
- *Restore page scrolling (usually or always disabled when a popup is open)* by calling *restorePageScroll()*

generateCustomRentalID()

- **Obsolete**

toast()

- **Probably obsolete**

del()

- **Probably obsolete**

openItemDetails(rowData)

- Open the Item Details popup given the *rowData*. The *rowData* includes the information on an item which is then put into the table rows in the Item Details popup

openRentalDetails()

- Open the Rental Details popup given the *rowData*. The *rowData* includes the information on an rental which is then put into the table rows in the Rental Details popup

openMakeRentalView()

- Imitates pressing the Scanning Mode button to open Scanning Mode

openAddItemsView()

- Opens the [Add Items popup](#).

openAddStudentsView()

- Opens the [Add Students popup](#).

openStudentDetails(fullRow)

- Opens the Student Details popup and populates the selected student's info. *fullRow* is static, not dynamic information that will be pressed based on the current row

makeARental(assetNumber)

- OBSOLETE

handleDelete()

- Gets the current menu which the user is in (Student Details, Item Details, etc) and then choose whether to delete the selected item based on that, and then deletes the follow student/item

dateStringToString(date)

- Converts

formatDateStringForDatabase(string date)

- Attempt to format *date* into YYYY-MM-DD (h/hh):mmAM/PM. Ex 2026-08 4:54PM and return the result as a string

renderItems(items)

- Clear all the items displayed in the “Items” tab and re-render them given the *items* variable. This variable is in the same structure as the output of *getItems()* on the backend

renderStudents(students)

- Clear all the items displayed in the “Students” tab and re-render them given the *students* variable. This variable is in the same structure as the output of *getStudents()* on the backend

renderRentals(students, rentals)

- Fills out the main screen with all of the active rentals, shows what the item is, the student that checked it out, and the due date and whether the item is overdue

openScanningMode()

- Opens the scanning mode in order to actually create the rental, it puts the cursor to be focused on the scan box, and then makes the barcode input available.

processScanningModeKeyUp()

- It detects whether the keys pressed, whether the delay between each letter is less than 70 milliseconds, and if there are 5 consecutive quick key presses, then it assumes that the scanner inputted a barcode.

submitScanningMode()

- Read the value the user has input into the <input> element with the ID scanning-mode-input-box. Check for empty input and other possible issues then try to create or return a rental depending on whether Scanning Mode is currently returning or creating rentals.

scanningModeFlashAndReset()

- Makes it so if any error happens during the rental process the whole screen goes red, and if it was a success then it goes green

getCodeType(string code)

- Determines whether a given code is a Student ID or an item Asset Tag, or not in the database

closeScanningMode()

- Hide the scanning mode popup, close the modal-dialog, remove the event listener that was keeping track of whether the user was typing or using a barcode scanner, and refresh the various tabs (So the Rental tab is up to date)

closeReportsView()

- Closes the report

scanningModeSelectMode(string mode)

- Select either 'create' mode or 'return' mode for determining whether *submitScanningMode()* should create or return rentals. Then refocus the user on the input box so they can keep scanning barcodes or typing them.

generateAndDownloadReport(string reportType, string timespanStart, string timespanEnd)

- This is related the 'Report View' popup which is disabled as of 2026-8-1
- Generate a PDF report of the specified *reportType()* by calling *generateReport()* on the backend

viewPastRentals()

- Show the user the URL to the database spreadsheet so they can manually look at past rentals

disablePageScroll()

- Prevents scrolling the main page content.
- Useful for when a popup is open and the user shouldn't be able to accidentally scroll the content behind the popup.

restorePageScroll()

Re-enable the user's ability to scroll on the dashboard and the various [Tab](#) pages

startLoadingAnimation()

- Basically whenever the rental process happens or the page is loading for the first time or the page is resized enough to be mobile size, it stops the user being able to scroll

endLoadingAnimation()

- Undos startLoadingAnimation()

showSection(string id, Event evt)

- Switch [tabs/section](#) to the section specified by ID (see the section linked previously for acceptable IDs)

1. How to read this codebase

A new maintainer will move faster if they read the code in layers instead of top to bottom.

- Start with Code.gs:21-40 to understand startup configuration and spreadsheet binding.
- Then read Code.gs:83 doGet to see how the app decides between setup.html and the main app.
- Next, read setup.html:145 submitURL so you understand first-run configuration.
- Then jump to index.html:1548 refresh and the render functions at index.html:2804, index.html:2918, and index.html:2997.
- Finish with index.html:3163 submitScanningMode and the rental functions in Code.gs:633 and Code.gs:885.

This codebase is organized around a few practical conventions rather than a formal framework.

- There is no bundler, build step, or typed API layer.
- Most UI behavior lives in one large script block inside index.html.
- Most server behavior lives in one Apps Script file, Code.gs.
- The spreadsheet is the persistence layer, so column order matters.
- Many UI buttons use inline onclick handlers, which means front-end function names must remain globally reachable.
- Data crosses the browser/server boundary as raw objects or JSON strings, not through shared types.

2. Runtime architecture

The system has four execution zones.

Zone

What lives there

Why it matters

Browser UI

index.html markup, popups, tables, client-side state, barcode/report PDF generation

This is where the user clicks buttons, sees tables, and triggers google.script.run calls.

Apps Script server

Code.gs top-level state and functions

This layer reads and writes Google Sheets, decides which page to serve, and sends emails.

Google Sheet database

Items, Students, and Rentals tabs

This is the source of truth for inventory, borrowers, and rental history.

External services

CDN libraries and MailApp email delivery

The UI depends on browser-loaded libraries for styling and PDF work, and the server depends on Google mail services for receipts.

The most important request path is the page load path.

```
User opens deployed web app URL
-> Apps Script runs doGet(e) in Code.gs:87
-> If spreadsheetURL is missing, return setup.html through getSetupPage()
-> Otherwise load index.html, split out the main script block, and inject that
JavaScript at runtime
-> Browser receives the page and runs refresh() in index.html:1548
-> refresh() fetches students, items, rentals, and spreadsheet URL through
google.script.run
-> renderStudents(), renderItem(), and renderRentals() rebuild the visible tables
```

The same bridge pattern is reused for almost every interaction: the browser collects input, calls a named Apps Script function, the server updates the spreadsheet, and the browser refreshes local state.

3. Repository map

File or resource

Primary responsibility

Best entry point

Code.gs

Main server-side logic. Owns configuration, page routing, spreadsheet reads and writes, rental creation and return, email generation, and report data.

Read Code.gs:21-40, Code.gs:83, Code.gs:198, Code.gs:502, Code.gs:536, Code.gs:633, and Code.gs:885 first.

index.html

Main web app UI. Owns page structure, client state, DOM rendering, modals, scanning mode, barcode generation, and report download behavior.

Start at index.html:815 for the visible layout and index.html:1440 for the main JavaScript block.

setup.html

First-run setup screen that collects the spreadsheet URL and stores it through the server.

Read setup.html:112-176, especially submitURL() at setup.html:145.

README.md

Short external overview and deployment notes.

Useful for orientation, but not detailed enough for maintenance.

Spreadsheet tabs

Operational data store for items, students, and rentals.

Treat the spreadsheet as part of the code contract, not just content.

4. Spreadsheet-driven data model

The code does not use schema files or model classes. Instead, it converts sheet rows directly into plain JavaScript objects.

Tab

Used for

Primary server readers/writers

Items

Inventory records and current checkout status

Read by getItems(); written by addItemToDatabase(), editItemsInDatabase(), deleteItemsFromInventory(), createRental(), and returnRentals().

Students

Borrower roster

Read by getStudents(); written by addStudentsToDatabase(), editStudentsInDatabase(), and deleteStudentsFromDatabase().

Rentals

Checkout and return history

Read by getRentals(); written by createRental() and returnRentals(); summarized by generateReport().

The most important object shapes are built in the read helpers.

Helper

Current object shape

Consumed by

Code.gs:502 getStudents()

id, classYear, period, name

index.html refresh(), renderStudents(), getCodeType(), createRental(), and student edit flows.

Code.gs:536 getItems()

roomArea, itemDescription, location, assetTagNumber, serialNumber, notes, status, rentalID, school
index.html renderItems(), getCodeType(), item modals, createRental(), returnRentals(), and report generation
helpers.

Code.gs:752 getRentals()

rentalID, itemID, status, checkedOutTo, dateCheckedOut, dateExpectedReturned, dateReturned, notes,
returnedBy, plus returnRental() method

index.html renderRentals(), createRental() for ID generation, returnRentals(), and generateReport().

Maintainer note: because these shapes are hand-built, any field change must be updated in all three places that care about it: the spreadsheet columns, the server-side row mapper, and the front-end rendering or form logic.

5. Backend guide: Code.gs

Code.gs is the server spine of the application. Everything that touches persistent data or Google services ends up here.

Subsystem

Key references

What it owns

Boot and configuration

Code.gs:21-40, 47, 61

Loads user properties, opens the spreadsheet, performs simple migration logic, and stores or resets the spreadsheet URL.

Routing and page serving

Code.gs:83, 87, 198

Returns setup or main HTML and exposes JSON snapshots to the browser.

Student and item CRUD

Code.gs:226, 273, 314, 372, 430, 456

Adds, edits, and deletes spreadsheet rows for the two core entity types.

Data mapping helpers

Code.gs:502, 536, 752

Transforms raw sheet rows into the objects the browser and server use.

Rental lifecycle

Code.gs:576, 633, 821, 885

Creates receipts, creates rentals, applies returns, and sends email confirmations.

Reports and utilities

Code.gs:999, 1133, 1137

Builds report payloads and exposes small lookup helpers to the browser.

5.1 Boot, configuration, and spreadsheet binding

The first important code runs before any function call. At Code.gs:21-40, the script loads User Properties, reads spreadsheetURL, and, if present, opens the spreadsheet immediately into the top-level datasheet variable.

- userPreferences is created with PropertiesService.getUserProperties().
- spreadsheetURL is read once at load time.
- datasheet is opened once at load time when spreadsheetURL is present.
- The startup block also inspects the Items sheet header row and adds a School column if it is missing.

This startup design explains why setup.html tells the user to reload after saving the spreadsheet URL: the top-level datasheet variable is bound during script initialization, so a fresh request is the safest way to ensure the server reopens the intended sheet.

The configuration helpers are small but central.

- Code.gs:47 setInventoryURL(url) stores the chosen spreadsheet URL and returns success or error.
- Code.gs:61 resetInventoryURL() clears the saved URL without deleting the property itself.
- Code.gs:42 del() deletes the property entirely. It is useful as a manual admin reset helper.

5.2 Routing and page assembly

Code.gs:87 doGet(e) is the main HTTP entry point. It is the first server function to understand when tracing a user request.

- If the request is for the default path, doGet decides whether to serve setup.html or index.html.
- If spreadsheetURL is missing, doGet returns getSetupPage() from Code.gs:83.
- If spreadsheetURL exists, doGet loads the raw text of index.html with ScriptApp.getResource("index").getDataAsString().

The next part is specific to this codebase and matters to maintainers: the main JavaScript in index.html is not served normally. Instead, doGet searches for the markers around index.html:1440, extracts the script body, base64-encodes it, and appends a runtime script tag that re-inserts the JavaScript on the client.

```
Code.gs:87 doGet(e)
  -> read raw index.html text
  -> split on "Start of js to remove and insert at runtime" markers
  -> extract the main script body
  -> base64 encode it
  -> append a tiny bootstrap script to the HTML response
  -> browser decodes it and appends a new script element at runtime
```

That workaround exists so the app can avoid a historical Apps Script one-line-script parsing issue. As a maintainer, preserve those marker comments and the general injection pattern unless you intentionally redesign page assembly.

Code.gs:198 getJSON(element) is the lightweight browser data API. It exposes three string-selected payloads: Student, Items, and Rentals. The function name is generic, but in practice it is the main read endpoint used by refresh() in the browser.

5.3 Student and item CRUD

The add and edit functions are thin wrappers around spreadsheet mutation. Their job is to validate identity rules, prepare row arrays, and write them back to the correct sheet ranges.

- Code.gs:226 addStudentsToDatabase(students) normalizes student IDs, derives classYear from the ID prefix, and appends rows to Students columns A:D.
- Code.gs:273 addItemsToDatabase(items) checks asset-tag uniqueness and appends rows to Items starting at column B.
- Code.gs:314 editItemsInDatabase(items) expects a JSON string, parses it, finds matching rows by asset tag, and overwrites the item range in place.
- Code.gs:372 editStudentsInDatabase(students) also expects a JSON string, finds rows by student ID, and overwrites columns A:D.
- Code.gs:430 deleteItemsFromInventory(assetTags) deletes matching rows from Items based on asset tag matches.
- Code.gs:456 deleteStudentsFromDatabase(ids) deletes matching student rows after skipping the header row.

The main maintainability lesson here is that add and edit paths do not share one contract. Adds accept live arrays or objects from the browser; edits accept JSON-encoded arrays. If you introduce a shared abstraction later, this difference is a good place to simplify.

5.4 Data mapping helpers

The read helpers are more important than they look because the rest of the app treats them as the canonical data contract.

- Code.gs:502 `getStudents()` reads the Students tab, removes the header row, and returns a dictionary keyed by student ID.
- Code.gs:536 `getItems()` reads the Items tab, removes the header row, and returns a dictionary keyed by asset tag.
- Code.gs:752 `getRentals()` reads the Rentals tab, removes the header row, and returns a dictionary keyed by rental ID.

A unique feature of `getRentals()` is that it attaches a `returnRental(returnedBy)` method onto each returned rental object. That means a rental record is not only data; it also knows how to write its own return back to the spreadsheet and reset the related item status.

This is one of the few places where the codebase mixes data and behavior in the same object. It is useful to know because `returnRentals()` relies on that method rather than rewriting the update logic itself.

5.5 Rental creation, return flow, and email receipts

Rental behavior is implemented as a coordinated update across two spreadsheet tabs plus optional email delivery.

The create flow is centered on Code.gs:633 `createRental(studentID, assetTag, dateToReturn)`.

```
Browser submitScanningMode() decides one code is a student and one is an item
-> browser calls createRental(studentID, assetTag, dateToReturn)
-> server checks that the student exists through getStudents()
-> server checks that the item status is In through getItems()
-> server computes the next rental ID from getRentals()
-> server appends a row to Rentals
-> server updates the matching Items row to status Out and stores the rental ID
-> server builds HTML email with buildRentalReceiptHTML()
-> server sends the email through MailApp
-> browser refreshes tables
```

The return flow is centered on Code.gs:885 `returnRentals(rentalsToReturn, returnedBy)`.

```
Browser submitScanningMode() in return mode calls returnRentals([itemCode], studentCode)
-> server loads all rentals with getRentals()
-> for each open rental whose itemID matches, call rental.returnRental(returnedBy)
-> returnRental() marks Rentals status Returned and stamps returned-by/date fields
-> returnRental() resets the matching item in Items to status In and clears rental ID
-> server builds one return receipt email with buildMultiRentalReceiptHTML()
-> server emails teacher and CCs the student
-> browser refreshes tables
```

The email helpers live at Code.gs:576 and Code.gs:821. They do not send mail by themselves; they only generate HTML bodies for `createRental()` and `returnRentals()`.

Important hardcoded configuration lives inside these rental paths.

- Teacher email comes from `Session.getActiveUser().getEmail()`.
- Student email is assembled as `s{studentID}@domain.org` at Code.gs:713 and Code.gs:906.
- Date formatting uses `Utilities.formatDate()` with the script time zone.

5.6 Reports and utility helpers

Code.gs:999 `generateReport(reportType, timespanStart, timespanEnd)` produces structured table data for the browser-side PDF export flow.

- itemPopularityUsageReport counts rentals within the requested time window and sorts items by usage.
- lostItemReport filters items by Lost status and tries to enrich the output from rental history.

Reports are delivered as JSON with head/body arrays so index.html can hand that object directly to jsPDF AutoTable.

The remaining small helpers are maintenance-oriented.

- Code.gs:1133 getSpreadsheetURL() returns the current spreadsheet URL so the browser can populate the Your Database and See Past Rentals links.
- Code.gs:1137 findMakeModelBasedOnAssetTag(assetTag) is used when opening item details; despite its name, it acts as a spreadsheet lookup helper tied to the current column layout.
- Code.gs:1104 doSomething() is a scratch or test helper, not a production request path.

6. Frontend guide: index.html

index.html mixes page markup, modal markup, utility elements, and one large JavaScript program. The fastest way to navigate it is to think in zones: layout first, then client state and lifecycle, then operational features such as add/edit flows, scanning, and PDFs.

Area

Reference

What lives there

Hidden utility elements

index.html:305

Canvas for barcode rendering and other non-visible helpers.

Scanning popup

index.html:311

Rental creation and return workflow UI.

Reports popup

index.html:407

Report date range and download controls.

Shared entity modal

index.html:490

Item details, student details, rental details, add-items, and add-students views.

Navigation and main sections

index.html:815, 939, 1063, 1233

Header, nav buttons, item list, rental list, and student list.

Loading overlay

index.html:1367

Global loading state while async calls are running.

Main JavaScript block

index.html:1440

All major client behavior from refresh() onward.

6.1 Startup lifecycle and global client state

The front-end lifecycle begins in two places: showSection() at index.html:1416 controls which major content section is visible, and refresh() at index.html:1548 repopulates all live data.

The main globals near index.html:1475 are the state anchors for the entire file.

- students: last fetched student dictionary from the server.
- items: last fetched item dictionary from the server.
- isDisabled: whether teacher assistant mode is active.

- checkedItems and checkedStudents: checkbox selection state for bulk actions.
- addItemRowIDs and addStudentsRowIDs: track dynamic rows inside the add-item and add-student modal views.
- scanningModeCodes: small in-memory queue of scanned codes while Rental Mode is active.
- reportsBeingGenerated: tracks in-flight report exports so duplicate requests can be avoided.

refresh() is the single most important browser function to understand.

```
index.html:1548 refresh()
  -> show loading overlay
  -> clear existing tbody content for items, students, and rentals
  -> fetch students through getJSON("Student")
  -> render students, then fetch rentals and render rentals
  -> fetch items through getJSON("Items") and render items
  -> fetch spreadsheet URL through getSpreadsheetURL()
  -> when all three payloads are ready, hide loading state and restore scroll
```

A maintainer should treat refresh() as the browser equivalent of re-syncing the app with the spreadsheet. Most mutations end by calling refresh() rather than trying to patch every visible row by hand.

6.2 Rendering pipeline

The render functions rebuild visible tables from the latest in-memory dictionaries.

- index.html:2804 renderItem(items) converts the item dictionary into table row HTML, updates checkedItems, and renders a status badge plus a details button for each row.
- index.html:2918 renderStudents(students) creates rows with checkboxes and details buttons, updating checkedStudents as it goes.
- index.html:2997 renderRentals(students, rentals) fetches items again from the server, then builds the outstanding-rental table with overdue calculations and a details button.

Each renderer is tightly coupled to both the front-end data shape and the expected DOM IDs. If you change field names or element IDs, update the corresponding renderer and any related modal logic in the same pass.

renderRentals() is worth special attention because it does one extra lookup pass: it fetches items again inside the function so it can decorate rentals with school/location context and overdue messaging.

6.3 Modal system and CRUD flows

The shared modal at index.html:490 is the UI shell for multiple tasks. Instead of rendering different dialog components, the file hides and shows different content blocks inside one modal container.

- openItemDetails() at index.html:2489 opens the item-detail view, decodes the row payload, asks the server for a supplemental lookup, and populates the table cells or select boxes.
- openStudentDetails() at index.html:2640 opens the student-detail view and fills in the current values.
- openRentalDetails() at index.html:2571 opens the rental-detail view for read-mostly inspection.
- openAddItemsView() and openAddStudentsView() switch the modal into bulk-add mode.
- saveModalEdits() at index.html:2312 is the central save dispatcher for the shared modal.

Bulk-add flows are split into row-generation and row-save phases.

- updateAddItemsPopupRows() at index.html:1883 appends editable rows for new item entry.
- saveAddItemsView() at index.html:2037 validates required asset tags, prevents duplicate entries within the modal, calls addItemToDatabase(), then generates a barcode PDF for the new items.
- updateAddStudentsPopupRows() at index.html:1963 appends editable student rows.

- saveAddStudentsView() at index.html:2148 validates IDs and numeric fields, prevents duplicates within the modal, and calls addStudentsToDatabase().

Delete behavior is centralized in handleDelete() at index.html:2677 for modal context, while deleteSelectedItems() and deleteSelectedStudents() handle bulk deletion from the tables.

One navigation-relevant detail: item and student edits travel through editItemsInDatabase() and editStudentsInDatabase() with JSON-stringified arrays. The browser does the stringification in saveModalEdits() before calling the server.

6.4 Barcode and report generation in the browser

Barcode and report exports are purely front-end PDF jobs. The server supplies data, but the browser creates the final files.

- generateBarcode() at index.html:1692 draws a CODE128 barcode onto the hidden canvas.
- generateBarcodePDF() at index.html:1707 creates a jsPDF document, repeatedly draws label text plus barcode images, and saves the file locally.
- generateItemBarcodes() and generateStudentBarcodes() gather selected records and pass them to generateBarcodePDF().
- generateAndDownloadReport() at index.html:3436 asks the server for report data, then feeds the response into jsPDF AutoTable and saves the finished PDF.

This division of labor is important for maintainers. If you want to change report contents, edit Code.gs:999. If you want to change report layout, filename, or PDF styling, edit index.html:3436.

6.5 Scanning mode flow

Scanning mode is the operational heart of the front end. It is also the clearest example of the browser making local decisions first, then delegating persistence to the server.

- openScanningMode() at index.html:3072 shows the popup, focuses the input box, wires key listeners, and prepares the UI for typed or scanned codes.
- processScanningModeKeyUp() at index.html:3124 watches typing speed to guess whether input is coming from a barcode scanner instead of a person.
- submitScanningMode() at index.html:3163 accumulates two codes, determines whether each code is a student or item by checking local state, and branches into create or return mode.
- getCodeType() at index.html:3376 classifies a code using the current in-memory students and items dictionaries.
- scanningModeFlashAndReset() at index.html:3337 handles the visual success or error feedback and resets the scanning UI.

Open Rental Mode

- > openScanningMode() focuses the shared input box
- > user scans or types two values
- > submitScanningMode() classifies each value with getCodeType()
- > if mode is create, prompt for return date and call createRental()
- > if mode is return, call returnRentals([itemCode], studentCode)
- > flash success or error state
- > refresh() reloads visible data from the server

Maintainer note: scanning mode depends on refresh() having already populated students and items. If those globals are stale or empty, getCodeType() cannot classify codes correctly.

6.6 Teacher assistant mode and UI capability reduction

The teacher assistant or self-checkout mode is not a second application state managed by routes or permissions. Instead, it is a client-side UI reduction mode triggered by the click handler attached at `index.html:2420`.

- The handler sets `isDisabled = true`.
- It opens Rental Mode immediately and tries to request fullscreen.
- It removes delete and save buttons from the DOM.
- It removes elements tagged with `edit-button` or `edit-button-like` IDs.
- It toggles the assistant-mode banner visibility.

Because the mode physically removes controls from the DOM, a page reload is the cleanest way to return to the full editing interface.

7. Setup page guide: `setup.html`

`setup.html` exists for one purpose: let a user bind their account to the correct spreadsheet before the main app loads.

Reference

Role

Notes

`setup.html:112-138`

Visible onboarding UI

Shows the template link, manual link, spreadsheet input, and submit button.

`setup.html:145 submitURL()`

Configuration submit handler

Normalizes the entered URL, validates it, calls `setInventoryURL()`, and tells the user to reload.

`setup.html:169-175`

Enter-key convenience handler

Lets the user press Enter instead of clicking Submit.

`submitURL()` is worth understanding because it explains the entire first-run setup contract.

```
User pastes Google Sheets URL
-> submitURL() strips any query string after ?
-> validate the URL contains docs.google.com/spreadsheets
-> call google.script.run.setInventoryURL(url)
-> server stores the URL in User Properties
-> UI tells the user to reload so the main app can bind datasheet on the next request
```

The setup page is coupled directly to `Code.gs:47 setInventoryURL()` and `Code.gs:87 doGet()`. If you change how configuration is stored, those three pieces must move together.

8. End-to-end code-flow maps

These short traces are useful when you are debugging by behavior rather than by file name.

8.1 First page load

```
Browser request
-> Code.gs:87 doGet(e)
-> setup.html or index.html response
-> index.html main script runs
-> index.html:1548 refresh()
-> Code.gs:198 getJSON("Student")
-> Code.gs:198 getJSON("Items")
-> Code.gs:198 getJSON("Rentals")
-> index.html renderStudents/renderItems/renderRentals
```

8.2 Importing students from CSV

```

User chooses CSV file in Students section
-> FileReader parses rows in index.html:1630-1689
-> client normalizes header presence and ID format
-> client calls Code.gs:226 addStudentsToDatabase(students)
-> server appends rows to Students tab
-> client calls refresh()

```

8.3 Adding items and generating barcodes

```

User opens Add Items modal
-> updateAddItemsPopupRows() creates editable rows
-> saveAddItemsView() validates and builds itemInfo
-> client calls Code.gs:273 addItemToDatabase(itemInfo)
-> server appends rows to Items tab
-> client builds barcode data set
-> generateBarcodePDF() saves local PDF
-> client refreshes tables

```

8.4 Creating a rental

```

openScanningMode()
-> submitScanningMode()
-> getCodeType() classifies student vs item
-> prompt for return date
-> Code.gs:633 createRental(studentID, assetTag, dateToReturn)
-> Rentals row appended, Items row updated, email sent
-> scanningModeFlashAndReset("success")
-> refresh()

```

8.5 Returning a rental

```

openScanningMode() in return mode
-> submitScanningMode()
-> Code.gs:885 returnRentals([itemCode], studentCode)
-> getRentals() returns objects with returnRental() method
-> rental.returnRental() updates Rentals and Items tabs
-> return receipt email is sent
-> refresh()

```

9. Function navigation reference

Use this section as a fast lookup table after you already understand the architecture.

9.1 Backend reference

Reference

Inputs and outputs

Used for

Code.gs:47 setInventoryURL(url)

Input: spreadsheet URL string. Output: success or error string.

Stores per-user spreadsheet configuration during setup.

Code.gs:61 resetInventoryURL()

Input: none. Output: none.

Clears the saved spreadsheet URL so the setup page can appear again.

Code.gs:83 getSetupPage()

Input: none. Output: HtmlOutput.

Returns setup.html for first-run users.

Code.gs:87 doGet(e)

Input: request event. Output: HtmlOutput.

Main web entry point for setup vs main app routing and runtime script injection.

Code.gs:198 getJSON(element)

Input: Student, Items, or Rentals. Output: JSON string.

Primary browser read endpoint used by refresh().
 Code.gs:226 addStudentsToDatabase(students)
 Input: array of student objects. Output: success or error string.
 Bulk-create student rows.
 Code.gs:273 addItemToDatabase(items)
 Input: array of item objects. Output: success or error string.
 Bulk-create item rows.
 Code.gs:314 editItemsInDatabase(items)
 Input: JSON string of item array. Output: error string or undefined-style success path.
 Update existing item rows in place.
 Code.gs:372 editStudentsInDatabase(students)
 Input: JSON string of student array. Output: error string or undefined-style success path.
 Update existing student rows in place.
 Code.gs:430 deleteItemsFromInventory(assetTags)
 Input: asset-tag array. Output: boolean-style result.
 Bulk-delete item rows.
 Code.gs:456 deleteStudentsFromDatabase(ids)
 Input: student-ID array. Output: boolean-style result.
 Bulk-delete student rows.
 Code.gs:502 getStudents()
 Input: none. Output: dictionary keyed by student ID.
 Builds the canonical in-memory student shape.
 Code.gs:536 getItems()
 Input: none. Output: dictionary keyed by asset tag.
 Builds the canonical in-memory item shape.
 Code.gs:576 buildRentalReceiptHTML(...)
 Input: receipt fields. Output: HTML string.
 Composes the checkout email body.
 Code.gs:633 createRental(studentID, assetTag, dateToReturn)
 Input: student ID, asset tag, return date string. Output: success or error string.
 Creates one rental and updates item status.
 Code.gs:752 getRentals()
 Input: none. Output: dictionary keyed by rental ID.
 Builds rental objects and attaches return behavior.
 Code.gs:821 buildMultiRentalReceiptHTML(...)
 Input: returned-rental summary data. Output: HTML string.
 Composes the return email body.
 Code.gs:885 returnRentals(rentalsToReturn, returnedBy)
 Input: item IDs plus returning student ID. Output: success or failure string.
 Processes returns, updates sheets, and emails receipt.
 Code.gs:999 generateReport(reportType, start, end)
 Input: report type plus date window. Output: JSON string.
 Supplies tabular report data to the browser PDF exporter.
 Code.gs:1133 getSpreadsheetURL()
 Input: none. Output: spreadsheet URL string.
 Lets the browser populate direct links into the spreadsheet.
 Code.gs:1137 findMakeModelBasedOnAssetTag(assetTag)
 Input: asset tag. Output: adjacent sheet value or not-found string.
 Lookup helper used while opening item details.

9.2 Frontend reference

Reference

Inputs and outputs

Used for

index.html:1416 showSection(id, evt)

Input: section ID and event-like target. Output: none.
Switches between Items, Rentals, and Students views.
index.html:1548 refresh()
Input: none. Output: none.
Reloads client state from the server and re-renders the page.
index.html:1692 generateBarcode(data)
Input: label text plus code. Output: barcode image data URL.
Renders one CODE128 barcode on the hidden canvas.
index.html:1707 generateBarcodePDF(info)
Input: array of label/code pairs. Output: browser-saved PDF.
Creates printable barcode sheets.
index.html:1771 generateItemBarcodes()
Input: none. Output: browser-saved PDF.
Exports selected item barcodes.
index.html:1792 deleteSelectedItems()
Input: none. Output: none.
Bulk-delete selected items through the server.
index.html:1867 generateStudentBarcodes()
Input: none. Output: browser-saved PDF.
Exports selected student barcodes.
index.html:1883 updateAddItemsPopupRows(count)
Input: row count. Output: none.
Adds editable rows to the add-items modal.
index.html:1963 updateAddStudentsPopupRows(count)
Input: row count. Output: none.
Adds editable rows to the add-students modal.
index.html:2037 saveAddItemsView()
Input: none. Output: none.
Validates new item entries, sends them to the server, and generates barcodes.
index.html:2148 saveAddStudentsView()
Input: none. Output: none.
Validates new student entries and sends them to the server.
index.html:2312 saveModalEdits()
Input: none. Output: none.
Dispatches saves for item, student, and bulk-add modal modes.
index.html:2489 openItemDetails(fullRow)
Input: base64-encoded item payload. Output: none.
Opens and populates the item-details modal.
index.html:2571 openRentalDetails(fullRow)
Input: base64-encoded rental payload. Output: none.
Opens the rental-details modal.
index.html:2606 openAddItemsView()
Input: none. Output: none.
Switches the shared modal into add-items mode.
index.html:2623 openAddStudentsView()
Input: none. Output: none.
Switches the shared modal into add-students mode.
index.html:2640 openStudentDetails(fullRow)
Input: student object. Output: none.
Opens and populates the student-details modal.
index.html:2677 handleDelete()
Input: none. Output: none.
Deletes the current modal entity if allowed.
index.html:2804 renderItem(items)
Input: item dictionary. Output: none.

Builds the item table rows.

index.html:2918 renderStudents(students)

Input: student dictionary. Output: none.

Builds the student table rows.

index.html:2997 renderRentals(students, rentals)

Input: student and rental dictionaries. Output: none.

Builds the outstanding-rental table.

index.html:3072 openScanningMode()

Input: none. Output: none.

Shows rental scanning UI and wires input listeners.

index.html:3163 submitScanningMode()

Input: current scanning input plus local state. Output: none.

Classifies codes and calls create or return server flows.

index.html:3337 scanningModeFlashAndReset(type, errorMessage)

Input: success or error state. Output: none.

Handles visual feedback for rental mode.

index.html:3376 getCodeType(code)

Input: scanned code. Output: item, student, notInDatabase, or conflict string.

Classifies a code using current client dictionaries.

index.html:3436 generateAndDownloadReport(type, start, end)

Input: report type and date range. Output: browser-saved PDF.

Requests report data from the server and renders the PDF.

index.html:3569 startLoadingAnimation() / 3574 endLoadingAnimation()

Input: none. Output: none.

Shows or hides the global loading overlay.

10. Change map for future maintainers

When you need to change behavior, start from the entry point that owns the user interaction, then follow the data path into the server and spreadsheet mapping helpers.

If you want to change...

Start here

Then inspect

Spreadsheet setup behavior

setup.html:145 submitURL() and Code.gs:47 setInventoryURL()

Code.gs:61 resetInventoryURL(), Code.gs:83 getSetupPage(), and Code.gs:87 doGet().

Which page loads first

Code.gs:87 doGet(e)

Code.gs:83 getSetupPage() and the top-level spreadsheetURL/datasheet initialization at Code.gs:21-40.

Main app layout or navigation

index.html section markers at 815, 939, 1063, and 1233

showSection() at index.html:1416 plus the render functions for the section you changed.

Item fields or item forms

index.html:2037 saveAddItemsView() and index.html:2489 openItemDetails()

Code.gs:273 addItemToDatabase(), Code.gs:314 editItemsInDatabase(), and Code.gs:536 getItems().

Student fields or student import behavior

index.html:1630 student file import handler and index.html:2148 saveAddStudentsView()

Code.gs:226 addStudentsToDatabase(), Code.gs:372 editStudentsInDatabase(), and Code.gs:502 getStudents().

Rental rules

index.html:3163 submitScanningMode()

Code.gs:633 createRental(), Code.gs:752 getRentals(), and Code.gs:885 returnRentals().

Email subject lines or receipt content

Code.gs:576 and Code.gs:821 receipt builders

Code.gs:633 createRental() and Code.gs:885 returnRentals() where MailApp.sendEmail() is called.

Barcode output

index.html:1692 generateBarcode() and index.html:1707 generateBarcodePDF()
index.html:1771 generateItemBarcodes() and index.html:1867 generateStudentBarcodes().
Report contents or report PDFs
Code.gs:999 generateReport()
index.html:3436 generateAndDownloadReport() and the report popup markup near index.html:407.
Teacher assistant mode behavior
index.html:2420 enableTeacherAssistantMode click handler
refresh() at index.html:1548 and any elements tagged with edit-button or related IDs.
A few habits will keep changes safe.

- Keep the script-injection markers around index.html:1440 intact unless you intentionally redesign the delivery model.
- When changing fields, update the spreadsheet, the server-side row mappers, and the front-end render or form code together.
- When renaming front-end functions, search the HTML for inline onclick references as well as JavaScript callers.
- After any behavioral change, re-test the full chain: load app, add entity, edit entity, create rental, return rental, and refresh tables.

11. Hardcoded configuration and derived configuration

This codebase mixes hardcoded values and spreadsheet-derived values. Knowing which is which makes maintenance much easier.

Type

Where it lives

Examples

Hardcoded in HTML

setup.html and index.html

Template spreadsheet link at setup.html:119, manual link at setup.html:130, school color map at index.html:1479, and major visible layout structure.

Hardcoded in server code

Code.gs

Student email domain at Code.gs:713 and Code.gs:906, report type names at Code.gs:999, and title strings for served pages.

Derived from spreadsheet data

Code.gs read helpers plus render functions

Student, item, and rental rows transformed by getStudents(), getItems(), getRentals(), then displayed by renderItems(), renderStudents(), and renderRentals().

Derived from user account context

Apps Script services

Session.getActiveUser().getEmail() for teacher email and User Properties for spreadsheet selection.

12. Final orientation for a new maintainer

If you are opening this code for the first time, the shortest path to confidence is:

- understand doGet(),
- understand refresh(),
- understand the spreadsheet row mappers,
- understand submitScanningMode(),
- and then trace createRental() and returnRentals().

Once those pieces make sense, the rest of the file is easier to navigate because almost every feature is a variation on the same pattern: collect input in the browser, call a named server function, update the spreadsheet, then refresh the browser state.